

자료구조 및 알고리즘

Chapter 4. 파이썬 기초

4.1 파이썬 언어의 특징

4.2 파이썬 사용 기초

4.3 프로그램 수행

4.4 패키지와 모듈

4.5 파이썬 개발 환경

1. 파이썬 언어의 특징

- 인터프리터 언어
- 융통성이 넘치는 언어
- 리스트를 다루는 기능이 뛰어난 언어
- 들여쓰기로 주종 관계를 표현하는 언어

(a) `>>> a = [3, 15, 20] # 리스트`

`>>> a.append(50)`

(b) `>>> print(a)`

`[3, 15, 20, 50]` ← 프린트된 결과

`>>> a.append("Lion")`

(c) `>>> print(a)`

`[3, 15, 20, 50, 'Lion']`

`>>> a[0] = "안녕"`

`>>> a[2] = 7.8`

(d) `>>> print(a)`

`['안녕', 15, 7.8, 50, 'List']`

그림 4-1 리스트 사용 예

`>>> s = 1; n = 11`

`>>> def arithSum(s, n):`

`>>> sum = 0`

`>>> for i in range(n):`

`>>> sum += i`

`>>> return sum`

`>>> print("Sum of", s, "thru", n-1, "is", arithSum(s, n))`

`Sum of 1 thru 10 is 55`

그림 4-2 들여쓰기 예

2. 파이썬 사용 기초

- 파이썬 코드의 수행

- 파이썬 코드는 .py 파일로 저장
- sample.py 수행 예(수행 프로그램 이름이 sample.py)

- 예1 `C:\Users\teacher\PycharmProjects\test> sample.py`
`Sum of 1 thru 10 is 55`

- 예2 `C:\Users\teacher\PycharmProjects\test> python sample.py`
`Sum of 1 thru 10 is 551`

■ 프린트

```
❶ >>> print('Hello!')
Hello!
>>> x = 77
>>> print('x is', x, "th")
x is 77 th
❷ >>> print("5's element is", end = " ")
>>> print("TEST")
5's element is TEST
```

그림 4-3 프린트 예 1

```
❶ >>> print('My', 'name', 'is', 'Tom')
My name is Tom
❷ >>> print('My', 'name', 'is', 'Tom', sep = '*')
My*name*is*Tom
>>> x = 77
>>> print("x is", x, "th")
x is 77 th
>>> print("x is ", x, "th", sep = "")
x is 77th
```

그림 4-4 프린트 예 2

separation

■ 수의 처리

```
>>> x = 5      # 정수
>>> y = 10.5    # 실수
❶ >>> z = x + y
>>> print('z is', z)
z is 15.5      # 정수와 실수를 더하면 실수가 된다
❷ >>> a = (int)(x + y) # 타입 캐스팅
>>> print('a is', a)
a is 15
```

그림 4-5 수 처리 예 1

```
❶ >>> x = (20 + 35) % 10 # 10으로 나눈 나머지
>>> print(x)
5
❷ >>> print(20 / 3)
6.666666666666667
❸ >>> print(20 // 3)
6
```

→ 다림.

그림 4-6 수 처리 예 2

■ 문자열

```
❶ >>> list = ['lion', 'in', 'oil']
    >>> print(list)
    ['lion', 'in', 'oil']
❷ >>> print("5's element")
    5's element
❸ >>> print('5"s element')
    5"s element
❹ >>> print(''5's element'')
    5's element
❺ >>> print("4\s element \ 6"s element")
    4's element \ 6"s element
```

Indented
이스케이프 문자.

그림 4-7 문자열 사용 예

- 제어 : 반복문

```
sum = 0  
for i in range(3, N+1):  
    sum += A[i]
```

(a) for

```
sum = 0; i = 3  
while i <= N:  
    sum += A[i]  
    i += 1
```

(b) while

그림 4-8 반복 제어문 사용 예: A[3...N]의 값 모두 더하기

■ 제어 : 조건문

(a)

```
if i < N:
    print("i is less than ", N)
```

(b)

```
if i < N:
    print("i is less than ", N)
else:
    print("i is not less than ", N)
    ...
```

(c)

```
if i < N:
    print("i < ", N)
elif i < K:
    print(N, " ≤ i < ", K)
    ...
else:
    print(K + " ≤ i")
```

그림 4-9 if 문 예

- 제어 : 순회1

for 변수 in 순회 가능한 집합:
 작업...

* 순회가능한 자료형.

range(), 리스트, 튜플, 딕셔너리, 집합

[] ()

{ ... }

{key: value}

■ 제어 : 순회1

→ $\text{range}(n) \rightarrow 0 \sim n-1$ 까지.

```
>>> for i in range(10):  
(a) >>>     print(i, end = ' ')  
0 1 2 3 4 5 6 7 8 9
```

```
>>> for i in range(4, 10, 2): # 시작:4, 끝: 10-1 = 9, 간격: 2  
(b) >>>     print(i, end = ' ')  
4 6 8
```

```
>>> for i in range(10, 4, -2): # 시작: 10, 끝: 4+1 = 5, 간격: -1  
(c) >>>     print(i, end = ' ')  
10 8 6
```

그림 4-10 for 루프의 순회 예 1

■ 제어 : 순회2

(a) 리스트 1

```
>>> for i in [0, 1, 2, 3]:
>>>     print(i, end = ' ')
0 1 2 3
```

(b) 리스트 2

```
>>> s = [0, 1, 2, 3]
>>> for i in s:
>>>     print(i, end = ' ')
0 1 2 3
```

(c) 튜플

```
>>> for i in (0, 1, 2, 3):
>>>     print(i, end = ' ')
0 1 2 3
```

불변.

(d) 딕셔너리

```
>>> d = {1:10, 2:20, 3:30, 10:40}
>>> for i in d:
>>>     print(i, ': ', d[i], end = ' ')
1 : 10, 2 : 20, 3 : 30, 10 : 40,
```

(e) 집합

```
>>> for i in {1, 2, 3, 10}:
>>>     print(i, end = ' ')
1 2 3 10
```

? (3 1 0 2)

(f) 문자열

```
>>> for i in 'testseq':
>>>     print(i, end = '-')
t-e-s-t-s-e-q-
```

불변

그림 4-11 for 루프의 순회 예 2

■ 불변 타입과 가변 타입

```
>>> a = 3
>>> print(id(a)) # id(a): 변수 a의 id
1425854368
```

```
❶ >>> a += 1
>>> print(id(a))
```

```
❷ 1425854384
```

→ 새롭게 할당됨

(정수는 불변 타입)

그림 4-12 불변 타입 예

```
>>> b = [1, 3, 5, 7]
>>> print(id(b))
32722696
```

```
❶ >>> b.append('test')
>>> print(id(b))
```

```
❷ 32722696
```

← 개변.

(리스트는 가변 타입)

그림 4-13 가변 타입 예

■ 리스트, 튜플, 딕셔너리, 집합

(a) 리스트

```
>>> a = [1, 3, 5, 7]
>>> print(a)
1 3 5 7
>>> for x in a
>>>     print(x, end = ':')
1: 3: 5: 7:
```

(b) 튜플

```
>>> t = (1, 3, 5, 7)
>>> print(t)
1 3 5 7
>>> for x in t
>>>     print(x, end = ':')
1: 3: 5: 7:
```

(c) 딕셔너리

```
>>> d = {1:10, 3:20, 5:30, 10:40}
>>> d[3] = 222
>>> d[50] = 55
>>> for i in d:
>>>     print(i, ': ', d[i], end = ', ')
1 : 10, 3 : 222, 5 : 30, 10 : 40, 50 : 55,
```

(d) 집합

```
>>> a = {1, 3, 5, 7}
>>> a.add(3) # 아무 영향 없음
>>> a.add(10) # 집합 a에 10을 추가
>>> print(a)
{1, 3, 5, 7, 10}
```

그림 4-15 리스트, 튜플, 딕셔너리, 집합 예

■ 복사호출, 참조호출, 할당호출

(a)	<pre>def twofold(a): a = 2 * a</pre>	(c)	<pre>def twofold(a): a = 2 * a return a</pre>
(b)	<pre>x = 5 # 정수 twofold(x) print(x) # 프린트 값: 5</pre>	(d)	<pre>x = 5 # 정수 x = twofold(x) print(x) # 프린트 값: 10</pre>

그림 4-16 복사호출 방식 언어에서 인자와 파라미터의 관계

(a)	<pre>def sample(a:int, b): a = 2 * a b.append('test')</pre>
(b)	<pre>x = 5 # 정수 y = [1, 2] # 리스트 sample(x, y) print(x) # 프린트 결과: 5 print(y) # 프린트 결과: [1, 2, 'test']</pre>

그림 4-17 불변 타입과 가변 타입 인자가 섞인 함수 호출 예

■ 클래스

- 생성자 + 메서드로 구성
- 생성자로 객체 초기화, 메서드는 클래스 내부 함수

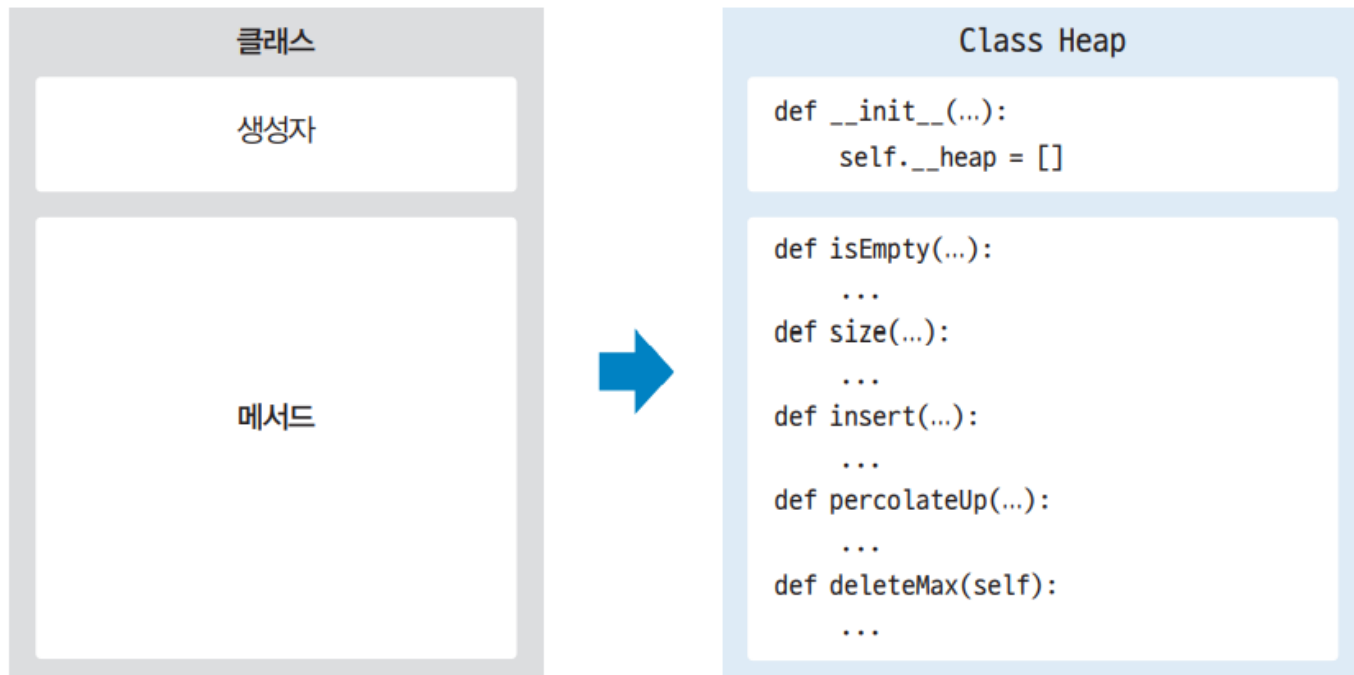


그림 4-18 클래스의 전형적 구조

■ 클래스 예

```
class Sample:
    def __init__(self):
        self.counter = 0
    def numEven(n):
        for k in range(n):
            if k % 2 == 0:
                self.counter += 1
```

```
class Sample:
    counter2 = 0
    def __init__(self):
        self.counter = 0
    def numEven(self, n):
        for k in range(n):
            if k % 2 == 0:
                self.counter += 1
                Sample.counter2 += 1
```

```
class Sample:
    def __init__(self, n):
        self.__table = [[] for i in range(n)]
        self.__numItems = 0
    def insert(newItem):
        ...
    def delete():
        ...
    def __percolateDown(i):
        ...
```

...? 오버라이딩 막는거.

그림 4-19 클래스 예

```
s = Sample()
print("# of even integers =", s.numEven(15))
```

```
>> # of even integers = 7
```

▪ 순회 가능 클래스

- 순회자(iterator)를 제공하는 클래스
- 내장 클래스: range, 리스트, 튜플, 딕셔너리, 집합, 문자열, ...

(a) 문자열1

```
>>> print ('s' in 'east')  
True
```

(b) 리스트1

```
>>> print(77 in [10, 20, 35, 50])  
False
```

(c) 문자열2

```
>>> for element in 'east':  
>>>     print(element, end = '-')  
e-a-s-t-
```

(d) 리스트2

```
>>> for x in [0, 1, 2, 3]:  
>>>     print(x, end = ' ')  
0 1 2 3
```

그림 4-20 키워드 in의 역할

■ 순회 예

- `for x in A` 호출 시 객체 A에서 `__iter__()` 호출

연장자도.

```
class L:
    def __init__(self):
        self.__head = ListNode("dummy", None)
        self.__numItems = 0
    def insert(self, i, newItem):
        ...
    def append(self, newItem):
        ...
    def __iter__(self): # 순회자 객체 생성
        return Iterator(self)
class Iterator: # 순회자
    def __init__(self, alist):
        self.iterPosition = alist.getNode(0) # 0번 노드
    def __next__(self):
        if self.iterPosition == None:
            raise StopIteration
        item = self.iterPosition.item
        self.iterPosion = self.iterPosion.next
        return item
```

연장자의 getNode.

▪ 순회 예

- for문을 한 번 돌 때마다 `__next__()` 수행

```
class L:
    def __init__(self):
        self.__head = ListNode("dummy", None)
        self.__numItems = 0
    def insert(self, i, newItem):
        ...
    def append(self, newItem):
        ...
    def __iter__(self): # 순회자 객체 생성
        return Iterator(self)
class Iterator: # 순회자
    def __init__(self, alist):
        self.iterPosition = alist.getNode(0) # 0번 노드
    def __next__(self):
        if self.iterPosition == None:
            raise StopIteration
        item = self.iterPosition.item
        self.iterPosion = self.iterPosion.next
        return item
```

▪ 순회 예

```
def count(self, x) -> int:
    cnt = 0
    curr = self.__head.next # 0번 노드
    while curr != None:
        if curr.item == x:
            cnt += 1
        curr = curr.next
    return cnt
```



순회자를 사용하면 간명해진다

```
def count(self, x) -> int:
    cnt = 0
    for a in self:
        if a == x:
            cnt += 1
    return cnt
```

3. 프로그램 수행

■ 프로그램 수행 예

```
class Test:
    def __init__(...):
        ...
    def isEmpty(...):
        ...
    ❶ print('클래스 내부 테스트')
    def insert(...):
        ❷ print('메서드 내부 테스트')
        ...
    ❸ print('클래스 외부 테스트')
```

그림 4-21 sample.py

```
class Heap: ↗ Counter=0
    def __init__(...):
        ...
class Test:
    def __init__(...):
        self.counter = 0 # 테스트 3
        ...
        counter = 0 # 테스트 4

    def count(self): # 테스트: 동적 변수와 정적 변수
        self.counter += 1
        Heap.counter += 1
        print("(self.counnter, Heap.counter) = (" , self.counter, ", ", Heap.counter, ")")
    def isEmpty(...):
        ...

h1 = Test()
h2 = Test()
h1.count()
h2.count()
h2.count()
```

그림 4-22 Test.py

■ 프로그램 수행 예

```
class Test:
    def __init__(...):
        self.counter = 0 # 테스트 3
    ...
    counter = 0          # 테스트 4
    def count(self):      # 테스트: 동적 변수와 정적 변수
        self.counter += 1
        Heap.counter += 1

        print("(self.counnter, Heap.counter) = (", self.counter, ", ", Heap.counter, ")")
    def isEmpty(...):
        ...
def start():
    h1 = Test()
    h2 = Test()
    h1.count()
    h2.count()
    h2.count()
if __name__ == "__main__":
    start()
```

→ 파이썬 파일은 해석과 소스코드 구분 없음으로.
main 코드를 구분하기 위함.

■ 프로그램 수행 예

```
class Test:
    def __init__(...):
        self.counter = 0 # 테스트 3
        ...
        counter = 0 # 테스트 4
    def count(self): # 테스트: 동적 변수와 정적 변수
        self.counter += 1
        Heap.counter += 1
        print("(self.counnter, Heap.counter) = (", self.counter, ",", Heap.counter, ")")
    def isEmpty(...):
        ...
def prepare4sample():
    ...

if __name__ == "sample":
    prepare4sample()
```

4. 패키지와 모듈

■ 패키지 사용 형식과 예

- ❶ `from 모듈이름 import *`
- ❷ `from 모듈이름 import 클래스이름`

- ❶ `from heap import *`
- ❷ `from DS.list.test import *`

■ 이 책의 파이썬 소스 코드 디렉터리 구조

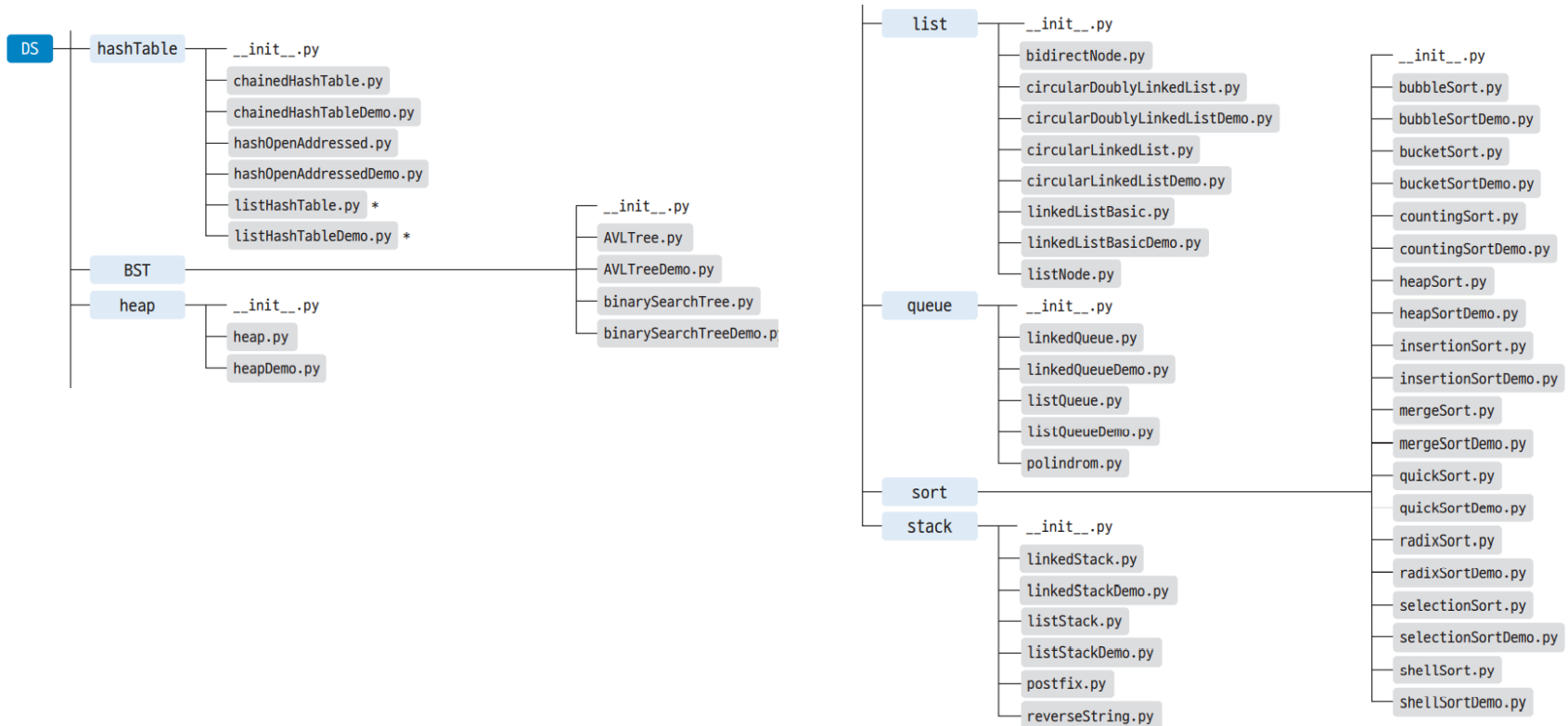


그림 4-23 이 책의 파이썬 소스 코드 디렉터리 구조(*는 참고용)

5. 파이썬 개발 환경

■ 파이참 수행 화면

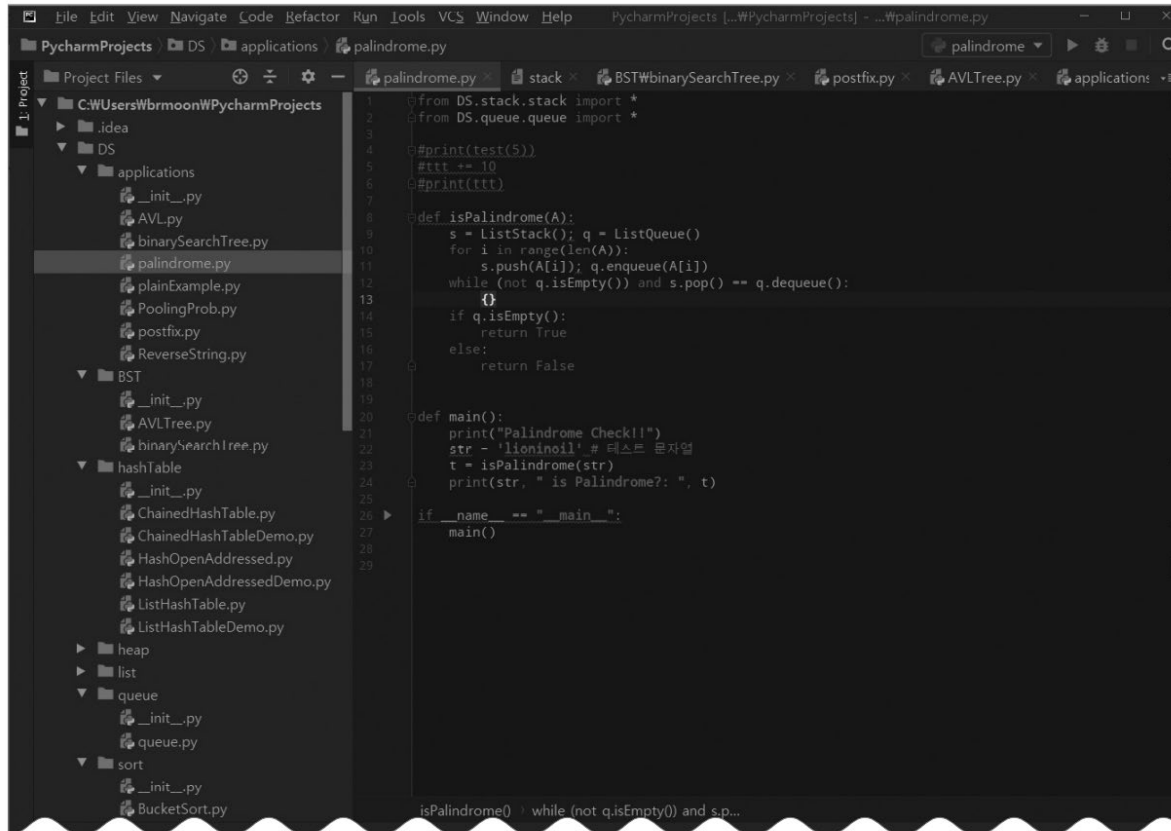


그림 4-24 파이참 수행 화면